KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

BANK ACCOUNT MANAGEMENT SYSTEM WITH DEPOSIT, WITHDRAWAL AND TRANSACTION HISTORY

Submitted by

Gowtham S

25BIT133

Under the Guidance of

MUGILAN R

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**BANK ACCOUNT MANAGEMENT SYSTEM WITH DEPOSIT, WITHDRAWAL AND TRANSACTION HISTORY"** submitted in partial fulfillment of the requirements for the course **Java Programming** is a **record of my original work** carried out under the guidance of **MUGILAN R**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimb

atore

_____

(GOWTHAM S)

CERTIFICATE

This is to certify that the project entitled **"BANK ACCOUNT MANAGEMENT SYSTEM WITH DEPOSIT, WITHDRAWAL AND TRANSACTION HISTORY"** is a **bona fide work** carried out by **GOWTHAM S B.Sc. IT 'B' (2025-2028)**, **INFORMATION TECHNOLOGY**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **JAVA PROGRAMMING.**

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The **Bank Account Management System with Deposit, Withdrawal and Transaction History** is a **Java-based application** developed to manage basic banking operations efficiently. In traditional systems, maintaining account details and transaction records manually can lead to errors, data loss, and difficulty in tracking financial activities.

This project provides a computerized solution for handling multiple bank accounts and performing essential operations such as account creation, deposit, withdrawal, and viewing transaction history. Each account is uniquely identified by an account number, and all transactions are recorded and maintained systematically for future reference.

The system is developed using **Core Java concepts** including arrays, loops, methods, and conditional statements. It is designed as a **menu-driven console application** that allows users to interact easily with the system.

The main objective of this project is to simplify banking operations, reduce manual errors, and provide an organized way of managing account and transaction data. The system ensures accurate balance updates and maintains a complete history of transactions for each user.

This project demonstrates the practical implementation of Java programming concepts in developing a simple banking application and can be further enhanced by integrating database support and graphical user interface features.

1. INTRODUCTION

The Bank Account Management System is a Java-based application designed to manage basic banking operations such as account creation, deposit, withdrawal, and transaction tracking. In traditional systems, managing account details manually can lead to errors, inefficiency, and difficulty in maintaining transaction history.

This project provides a simple and efficient computerized solution for handling multiple bank accounts and their transactions. The system allows users to create accounts, perform banking operations, and view transaction history in an organized manner.

The application is developed using Core Java concepts such as arrays, loops, methods, and conditional statements.

1.1 Objective

The main objectives of this project are:

- To develop a Java-based banking system
- To manage multiple user accounts efficiently
- To perform deposit and withdrawal operations
- To maintain transaction history for each user
- To reduce manual errors in banking operations
- To demonstrate the use of Core Java programming concept

1.2 Overview

The Bank Account Management System with Deposit, Withdrawal and Transaction History is a menu-driven application developed using the Java programming language. The system provides a digital platform for managing bank accounts and performing basic banking operations efficiently.

The system allows users to create new accounts, deposit and withdraw money, and maintain transaction history for each account. Each account is assigned a unique account number, and all financial transactions are recorded systematically. The program helps users easily track their account balance and view past transactions whenever required.

The application simplifies banking operations by reducing manual calculations and minimizing errors in maintaining account records. It ensures accurate balance updates after every transaction and provides a clear record of all activities performed by the user.

This project demonstrates the practical application of Java programming concepts in developing a simple banking system suitable for learning purposes and small-scale financial management.

1.3 Feature

The Bank Account Management System with Deposit, Withdrawal and Transaction History includes the following main features:

• **Account Creation** – Allows users to create a new bank account by entering basic details such as account holder name and initial deposit.

• **User Login System** – Enables users to access their account securely using a unique account number.

• **Deposit Functionality** – Allows users to deposit money into their account and updates the balance instantly.

• **Withdrawal Functionality** – Allows users to withdraw money with proper validation to prevent overdrawing.

• **Transaction Management** – Records every transaction (deposit and withdrawal) with amount, balance, and notes.

• **Transaction History** – Maintains a complete history of all transactions for each user for easy tracking.

• **Account Listing** – Displays all existing accounts with account number, holder name, and balance.

• **User-Friendly Interface** – Provides a simple menu-driven console interface for easy navigation and operation.

• **Efficient Data Management** – Organizes account and transaction data systematically using arrays for quick access and updates.

2. PROGRAM ANALYSIS

Program analysis is an important phase in software development where the problem is studied in detail and the requirements of the system are identified. This stage helps in understanding how the system should function and what features are required to perform banking operations effectively.

The analysis phase focuses on identifying the limitations of traditional or manual banking methods, such as maintaining records on paper or performing calculations manually. These methods can lead to errors, difficulty in tracking transactions, and inefficient management of account data.

To overcome these issues, a computerized solution is proposed in the form of the Bank Account Management System. This system improves efficiency by automating account operations such as deposit, withdrawal, and transaction tracking. It ensures accuracy in balance calculations and provides a structured way to manage account information and transaction history.

2.1 Problem Definition

In many small banking systems or personal financial management scenarios, account details and transactions are maintained manually using notebooks or simple records. This traditional method creates several problems.

Some common problems include:

- Difficulty in maintaining multiple account records
- Risk of losing transaction details
- Time-consuming process to track account balance
- Errors in manual calculations
- Lack of organized transaction history

Manual systems also increase the chances of mistakes and make it difficult to track previous transactions efficiently. Therefore, there is a need for an automated system that can manage bank accounts and transactions accurately and efficiently.The Bank Account Management System with Deposit, Withdrawal and Transaction History is designed to solve these problems by providing a computerized solution for managing account details and financial transactions.

2.2 Project Overview

The Bank Account Management System is a Java-based application developed to manage multiple bank accounts and perform basic banking operations efficiently.

The system allows users to create new accounts, deposit money, withdraw funds, and view transaction history. Each account is uniquely identified by an account number, and all transactions are recorded systematically. The system ensures that users can access their account details and transaction records whenever required, which helps in maintaining transparency and accuracy.

The system includes the following functionalities:

- Creating new bank accounts
- Logging into existing accounts
- Depositing money into accounts
- Withdrawing money with proper validation
- Viewing transaction history
- Listing all available accounts

This project is implemented using Core Java programming concepts and demonstrates how a simple banking system can be developed for efficient financial management.

2.3 Scope and Limitations

Scope

The scope of the Bank Account Management System with Deposit, Withdrawal and Transaction History includes managing basic banking operations efficiently using a computerized system.

The system is capable of creating and managing multiple user accounts. It allows users to perform deposit and withdrawal operations and maintains a detailed transaction history for each account. The system ensures accurate balance updates after every transaction and provides an organized way to store account details. It also demonstrates the use of Core Java programming concepts such as arrays, methods, loops, and conditional statements.

The system can be used in:

Small-scale banking environments
Educational purposes for learning Java programming
Personal financial management systems

Limitations

- The system does not include advanced banking features such as interest calculation or loan management.
- It does not provide authentication using passwords, as it only uses account numbers for login.
- Data is not stored permanently since no database is used, so all information is lost when the program is closed.
- The system is console-based and does not include a graphical user interface.
- The number of users and transactions is limited due to the use of fixed-size arrays.

3. SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before

implementation. In this phase, the system is divided into different modules and the workflow of the application is defined.

The Bank Account Management System with Deposit, Withdrawal and Transaction History is designed as a menu-driven application where users can create accounts, perform banking operations, and view transaction details efficiently.

3.1 Modules

The main modules of the Bank Account Management System are explained below:

1. Account Creation Module

This module is responsible for creating new bank accounts and storing account details.

Example Data:

Account Number

Account Holder Name

Initial Balancer

This helps in maintaining basic account records.

2. Login Module

This module allows users to access their account using a unique account number.

It verifies the account details and provides access to account operations.

3. Deposit Module

This module allows users to withdraw money from their account.

It checks for sufficient balance before processing the transaction

4. Transaction Management Module

This module stores all transaction details such as deposit and withdrawal operations.

Example Data:

Transaction Type

Transaction Amount

Updated Balance

Transaction Note

This helps in maintaining a complete history of account activities.

5. Transaction History Module

This module displays all past transactions of the user in an organized manner.

It allows users to review their account activity easily.

6. Account Listing Module

This module displays all existing accounts with their details.

It helps in managing multiple users in the system.

7. Exit Module

This module allows the user to exit the application safely after completing operations.

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the

system..

For the Bank Account Management System with Deposit, Withdrawal and Transaction History, a structured development approach was followed to ensure that the system meets the requirements and functions efficiently.

The development process includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

4.1. Problem Analysis

In this phase, the problems in manual banking systems were studied.

Major problems include:

• Manual record management

• Difficulty in tracking transactions

• Risk of data loss

• Errors in balance calculation

• Time-consuming operations

Based on these problems, an automated system was designed.

4.2. System Design

The system was designed by dividing the application into modules such as:

• Account Creation Module

• Login Module

• Deposit Module

• Withdrawal Module

• Transaction History Module

• Account Listing Module

• Exit Module

This structure helps the program run efficiently.

4.3. Program Development

The program was developed using the Java programming language.

The development steps included:

• Creating arrays to store account and transaction details

• Implementing menu-driven logic

• Writing methods for deposit and withdrawal operations

• Accepting user input using Scanner

• Displaying account details and transaction history

Core Java concepts such as loops, conditional statements, arrays, and methods were used.

4.4. Testing and Validation

Testing ensures that the program works correctly.

Example Test Cases:

Test Case	**Expected Result**
Create Account	Account created successfully
Deposit Money	Balance increases correctly
Withdraw Money	Balance decreases correctly
Invalid Withdrawal	Error message displayed
View Transaction History	Transactions displayed correctly

4.5. Result and Documentation

After successful testing, the system performs all banking operations correctly and maintains accurate transaction records.

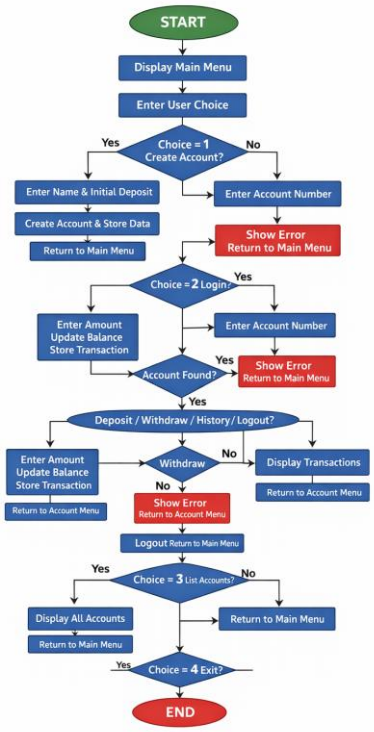
Screenshots of program execution are included in the documentation to show input and output result

4.6. Algorithm

Algorithm for the Bank Account Management System:

1. Start the program
2. Display the main menu
3. Select an option
4. If creating account, store account details
5. If login, verify account number
6. Perform deposit or withdrawal
7. Update balance and store transaction
8. Display transaction history when required
9. Repeat operations until user exits

4.7. Flowchart



5. SOURCE CODE

```
import java.util.Scanner;

public class BankSystem {

    static Scanner sc = new Scanner(System.in);

    static final int MAX_USERS = 10;
    static final int MAX_TXN   = 100;

    static String[] accountNumber  = new String[MAX_USERS];
    static String[] accountHolder  = new String[MAX_USERS];
    static double[] balance        = new double[MAX_USERS];
    static int      totalUsers     = 0;

    static String[][] txnType    = new String[MAX_USERS][MAX_TXN];
    static double[][] txnAmount  = new double[MAX_USERS][MAX_TXN];
    static double[][] txnBalance = new double[MAX_USERS][MAX_TXN];
    static String[][] txnNote    = new String[MAX_USERS][MAX_TXN];
    static int[]      txnCount   = new int[MAX_USERS];

    static int currentUser = -1;

    public static void main(String[] args) {

        System.out.println("==================================");
        System.out.println("  BANK ACCOUNT MANAGEMENT SYSTEM  ");
        System.out.println("==================================");

        int choice = 0;
```

```
    while (choice != 4) {
        System.out.println("\n-----------------------------------");
        System.out.println("           MAIN MENU");
        System.out.println("-----------------------------------");
        System.out.println("1. Create New Account");
        System.out.println("2. Login to Account");
        System.out.println("3. List All Accounts");
        System.out.println("4. Exit");
        System.out.println("-----------------------------------");
        System.out.print("Enter choice: ");
        choice = sc.nextInt();

        if (choice == 1) {
            createAccount();
        } else if (choice == 2) {
            loginMenu();
        } else if (choice == 3) {
            listAccounts();
        } else if (choice == 4) {
            System.out.println("\nThank you for banking with us. Goodbye!");
        } else {
            System.out.println("Invalid choice. Try again.");
        }
    }
}

static void createAccount() {
    if (totalUsers >= MAX_USERS) {
        System.out.println("Maximum number of accounts reached!");
        return;
    }
```

```
    sc.nextLine();
    System.out.print("\nEnter account holder name: ");
    String name = sc.nextLine().trim();

    System.out.print("Enter initial deposit: Rs.");
    double initial = sc.nextDouble();

    if (initial < 0) {
        System.out.println("Initial deposit cannot be negative.");
        return;
    }

    String accNo = "ACC" + (1001 + totalUsers);

    accountNumber[totalUsers] = accNo;
    accountHolder[totalUsers] = name;
    balance[totalUsers]       = initial;
    txnCount[totalUsers]      = 0;

    addTransaction(totalUsers, "DEPOSIT", initial, initial, "Initial deposit");

    System.out.println("\nAccount created successfully!");
    System.out.println("Account Number : " + accNo);
    System.out.println("Account Holder : " + name);
    System.out.println("Opening Balance: Rs." + initial);

    totalUsers++;
}

static void loginMenu() {
    if (totalUsers == 0) {
        System.out.println("No accounts exist yet. Please create one first.");
```

```
        return;
    }

    System.out.print("\nEnter Account Number (e.g. ACC1001): ");
    String input = sc.next().trim().toUpperCase();

    int idx = -1;
    for (int i = 0; i < totalUsers; i++) {
        if (accountNumber[i].equalsIgnoreCase(input)) {
            idx = i;
            break;
        }
    }

    if (idx == -1) {
        System.out.println("Account not found!");
        return;
    }

    currentUser = idx;
    System.out.println("\nWelcome back, " + accountHolder[currentUser] + "!");
    System.out.println("Current Balance: Rs." + balance[currentUser]);

    accountMenu();
}

static void accountMenu() {
    int choice = 0;
    while (choice != 4) {
        System.out.println("\n-----------------------------------");
        System.out.println(" Account: " + accountNumber[currentUser]
                + " | " + accountHolder[currentUser]);
```

```
            System.out.println("-----------------------------------");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
            System.out.println("3. Transaction History");
            System.out.println("4. Logout");
            System.out.println("-----------------------------------");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            if (choice == 1) {
                deposit();
            } else if (choice == 2) {
                withdraw();
            } else if (choice == 3) {
                showHistory();
            } else if (choice == 4) {
                System.out.println("Logged out. Goodbye, " + accountHolder[currentUser] + "!");
                currentUser = -1;
            } else {
                System.out.println("Invalid choice. Try again.");
            }
        }
    }

    static void deposit() {
        System.out.print("\nEnter deposit amount: Rs.");
        double amount = sc.nextDouble();

        if (amount <= 0) {
            System.out.println("Amount must be greater than 0.");
            return;
        }
```

```
    balance[currentUser] += amount;
    sc.nextLine();
    System.out.print("Enter note (or press Enter to skip): ");
    String note = sc.nextLine().trim();
    if (note.isEmpty()) note = "Deposit";

    addTransaction(currentUser, "DEPOSIT", amount, balance[currentUser], note);
    System.out.println("Deposited Rs." + amount + " successfully!");
    System.out.println("New Balance: Rs." + balance[currentUser]);
}

static void withdraw() {
    System.out.print("\nEnter withdrawal amount: Rs.");
    double amount = sc.nextDouble();

    if (amount <= 0) {
        System.out.println("Amount must be greater than 0.");
        return;
    }

    if (amount > balance[currentUser]) {
        System.out.println("Insufficient balance! Available: Rs." + balance[currentUser]);
        return;
    }

    balance[currentUser] -= amount;
    sc.nextLine();
    System.out.print("Enter note (or press Enter to skip): ");
    String note = sc.nextLine().trim();
    if (note.isEmpty()) note = "Withdrawal";
```

```
    addTransaction(currentUser, "WITHDRAWAL", amount, balance[currentUser], note);
    System.out.println("Withdrawn Rs." + amount + " successfully!");
    System.out.println("New Balance: Rs." + balance[currentUser]);
}

static void showHistory() {
    int u = currentUser;
    System.out.println("\n==================================");
    System.out.println("      TRANSACTION HISTORY");
    System.out.println("  " + accountHolder[u] + " | " + accountNumber[u]);
    System.out.println("==================================");

    if (txnCount[u] == 0) {
        System.out.println("No transactions yet.");
        return;
    }

    for (int i = 0; i < txnCount[u]; i++) {
        System.out.println((i + 1) + ". " + txnType[u][i]
                + " | Rs." + txnAmount[u][i]
                + " | Balance: Rs." + txnBalance[u][i]
                + " | " + txnNote[u][i]);
    }

    System.out.println("----------------------------------");
    System.out.println("Current Balance: Rs." + balance[u]);
    System.out.println("==================================");
}

static void listAccounts() {
    if (totalUsers == 0) {
        System.out.println("\nNo accounts found.");
```

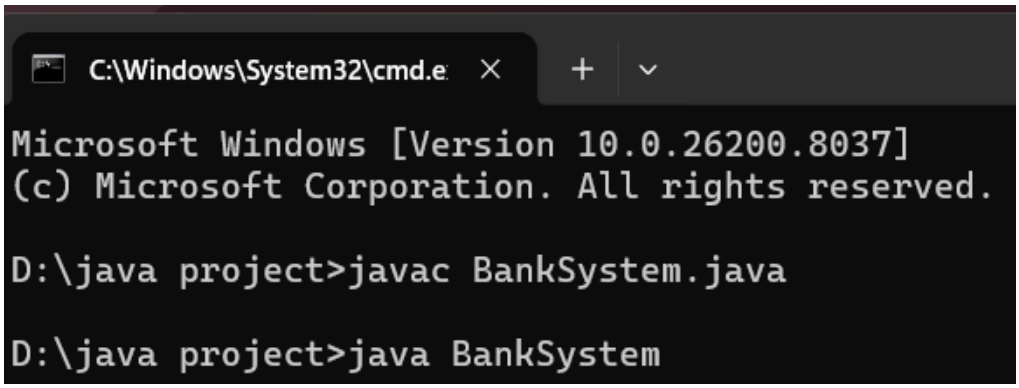
```
            return;
        }

        System.out.println("\n==================================");
        System.out.println("         ALL ACCOUNTS");
        System.out.println("==================================");
        for (int i = 0; i < totalUsers; i++) {
            System.out.println((i + 1) + ". " + accountNumber[i]
                    + " | " + accountHolder[i]
                    + " | Rs." + balance[i]);
        }
        System.out.println("==================================");
    }

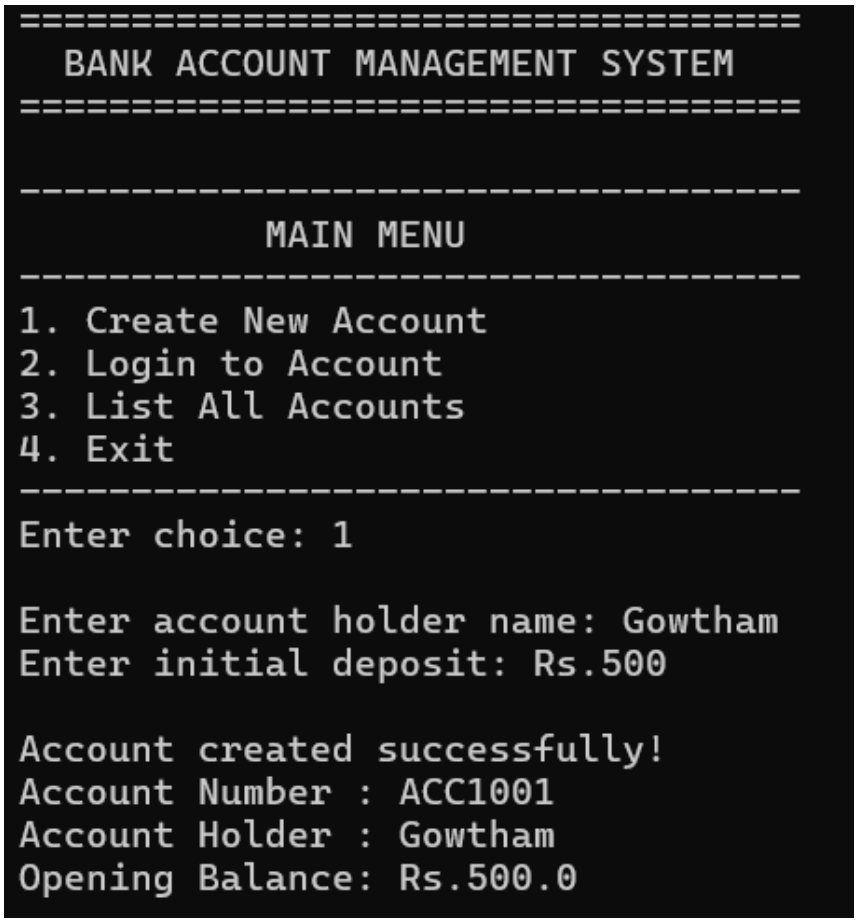
    static void addTransaction(int userIdx, String type, double amount, double bal, String note) {
        int t = txnCount[userIdx];
        txnType[userIdx][t]    = type;
        txnAmount[userIdx][t]  = amount;
        txnBalance[userIdx][t] = bal;
        txnNote[userIdx][t]    = note;
        txnCount[userIdx]++;
    }
}
```

6. INPUT AND OUTPUT SCREENS

COMPILATION SCREEN:



INPUT SCREEN:



OUTPUT SCREEN:

```
----------------------------------
 Account: ACC1001 | Gowtham
----------------------------------
1. Deposit
2. Withdraw
3. Transaction History
4. Logout
----------------------------------
Enter choice: 2

Enter withdrawal amount: Rs.200
Enter note (or press Enter to skip):
Withdrawn Rs.200.0 successfully!
New Balance: Rs.300.0

----------------------------------
 Account: ACC1001 | Gowtham
----------------------------------
1. Deposit
2. Withdraw
3. Transaction History
4. Logout
----------------------------------
Enter choice: 3

==================================
      TRANSACTION HISTORY
  Gowtham | ACC1001
==================================
1. DEPOSIT | Rs.500.0 | Balance: Rs.500.0 | Initial deposit
2. WITHDRAWAL | Rs.200.0 | Balance: Rs.300.0 | Withdrawal
----------------------------------
Current Balance: Rs.300.0
==================================
```

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Bank Account Management System with Deposit, Withdrawal and Transaction History was developed to simplify basic banking operations. The system allows users to create accounts, perform transactions, and maintain transaction history in a digital format.

Core Java programming concepts such as arrays, loops, methods, and conditional statements were applied during the development of this project.

Testing confirmed that the system works correctly by performing accurate deposit and withdrawal operations while maintaining proper transaction records. The system ensures organized management of account details and reduces the chances of manual errors.

7.2 Conclusion

The Bank Account Management System successfully demonstrates the practical use of Core Java programming in developing a basic banking application.

The system helps in managing account details, performing transactions, and maintaining transaction history efficiently. It reduces manual work and ensures accurate financial calculations.

Future improvements may include:

- Database integration for permanent data storage
- Password-based authentication for better security
- Graphical user interface for improved user experience
- Expansion of features such as interest calculation and fund transfer

8. REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – Java: The Complete Reference

Websites

1. https://www.oracle.com/java

2. https://www.geeksforgeeks.org

3. https://www.w3schools.com

4. https://www.javatpoint.com